

Sea Ice Deep Fusion

Pipeline Guide: Data Flow, Scripts, and Training Stages

Multimodal fusion of Sentinel-2 optical imagery and ICESat-2 ATL03 photon-altimetry data

Pipeline Overview

Stage	Description	Key Scripts / Outputs
Stage 1	Raw Data Preparation	crop_all.py, crop_csv.py
Stage 2	Ground-Truth Mask Generation	segment_all.py
Stage 3	BiLSTM Hyperparameter Sweep	lstm_sweep.ipynb (21 configurations)
Stage 4	Fusion Model Training	deep_fusion.ipynb, fusion_late_unet.ipynb, fusion_hybrid_unet.ipynb
Stage 5	Evaluation and Reproduction	test_metrics.json, confmat.png, loss_curve.png

This document describes the pipeline only: inputs, scripts, parameters, and output artifacts for each stage. Results tables, confusion matrices, and ablation study comparisons are provided in project_summary.pdf.

Stage 1: Raw Data Preparation

Aligned 128×128 pixel patches are extracted from Sentinel-2 GeoTIFF scenes, each centered on a labeled ICESat-2 ground point. The resulting dataset pairs optical imagery with ground-truth positions, constituting the primary input for all subsequent supervised training stages.

Input Data

IS2_Corrected_data/	ATL03 photon CSV files, one per beam per tile. Each row records a labeled 10-m along-track segment with pixel coordinates (pix_x, pix_y) projected into the corresponding Sentinel-2 tile.
S2_tiff/	Sentinel-2 Level-1C GeoTIFF scenes at 10-m resolution for three tiles: T02CNA, T02CNC (training), T03CWT (held-out evaluation). Not version-controlled; must be supplied locally.

Implementation Scripts

crop_csv.py	Single-pair utility. Processes all rows in one CSV against one GeoTIFF. Writes PNG patches to outputs/{csv_stem}__{tif_stem}/ together with a manifest.csv tracing each patch to its source CSV row, pixel coordinates, and class label.
crop_all.py	Batch implementation. Iterates all six CSV/GeoTIFF job pairs (two beams × three tiles) in a single execution. Each GeoTIFF is loaded once and reused across its two beams. All patches are written to outputs/.

Configuration Parameters

Patch size	128 × 128 pixels
Center alignment	(pix_x, pix_y) from the CSV record placed at the patch center
Edge handling	Zero-padded to maintain a constant 128×128 output at image borders
Output filename	row{N}_{date}_{tile}_{beam}.png (crop_all.py)
Tiles processed	T02CNA (gt1r, gt2r), T02CNC (gt1r, gt2r), T03CWT (gt1r, gt2r)

Execution

```
python crop_all.py      # batch: all 6 CSV/GeoTIFF pairs -> outputs/
python crop_csv.py      # single-pair diagnostic run
```

Output Artifacts

- outputs/ : flat directory of PNG patches, one per labeled CSV row
- Each patch: 128×128 RGB, uint8, labeled point at center

Stage 2: Ground-Truth Mask Generation

Each RGB patch is converted into a three-class per-pixel segmentation mask via an HSV color-thresholding pipeline

incorporating shadow and cloud removal. The resulting masks constitute the ground-truth labels for all classifier training and evaluation.

Implementation Script

segment_all.py	Applies <code>shadow_cloud_removal()</code> followed by <code>color_segmentation()</code> to every PNG in <code>outputs/</code> and writes the result to <code>outputs_segmented/</code> under the same filename. Files with an existing segmented counterpart are skipped, rendering the script safely resumable.
-----------------------	--

Processing Algorithm

- Step 1. Open-water isolation: pixels where HSV V is at most 30 are assigned to the open-water class and replaced with white to preclude influence on subsequent processing steps.
- Step 2. Shadow and cloud removal: a background model is constructed via grayscale dilation and median blur (kernel size 155); foreground is isolated using absolute difference and Otsu thresholding; the result is normalized to [0, 255] and highlights are truncated at 235.
- Step 3. Color segmentation: the processed image is re-classified into three classes via HSV value thresholds. Thick ice: V in [205, 255]. Thin ice: V in [31, 204]. Open water: V in [0, 30].
- Step 4. Pixel labeling: thick ice is assigned red (255, 0, 0); thin ice is assigned blue (0, 0, 255); open water is assigned green (0, 255, 0).

HSV Thresholds

Thick ice	H [0, 185], S [0, 255], V [205, 255]; label: red (255, 0, 0)
Thin ice	H [0, 185], S [0, 255], V [31, 204]; label: blue (0, 0, 255)
Open water	H [0, 185], S [0, 255], V [0, 30]; label: green (0, 255, 0)

Execution

```
python segment_all.py      # input: outputs/    output: outputs_segmented/
```

Output Artifacts

- `outputs_segmented/` : three-class RGB segmentation masks, same filenames as input patches
- Not version-controlled; regenerate from `outputs/` as required

Stage 3: BiLSTM Hyperparameter Sweep

A standalone bidirectional LSTM (BiLSTM) photon-branch classifier is trained across 21 hyperparameter configurations to identify the parameter set that maximizes mean Intersection-over-Union (mIoU) on the held-out evaluation tile. The checkpoint from the optimal configuration is subsequently transferred to Stage 4.

Implementation Script

lstm_sweep.ipynb

Executes all 21 configurations sequentially. Each configuration is trained to convergence, evaluated on the held-out tile T03CWT, and its artifacts are written to `archive/runs/lstm_sweep/{config_name}/`. Aggregated results across all configurations are recorded in `archive/runs/lstm_sweep/sweep_results.csv`.

Input Feature Specification

ATL03 photon records are aggregated to 10-meter along-track segments. A sliding window of consecutive segments is centered on each labeled point; eight scalar features are extracted per segment:

h_cor_mean	Mean corrected photon height (m)
h_cor_med	Median corrected photon height (m)
h_diff	Mean minus median height (within-segment asymmetry)
rel_height_min_elev	Mean height relative to the per-track minimum elevation
height_sd	Standard deviation of photon heights
pcnth_mean	Mean photon-count height
pcnt_mean	Mean photon count
bcnt_mean / brate_mean	Mean background photon count and background rate

All 21 Sweep Configurations

One parameter is varied at a time relative to the baseline. The highlighted row identifies the configuration achieving the highest test mIoU.

Config	Focal alpha	gamma	hidden	lr	dropout	seq	seed
baseline	[0.05, 0.45, 0.60]	2.0	48	8.886e-4	0.4	5	42
alpha [0.02,0.44,0.54]	[0.02, 0.44, 0.54]	2.0	48	8.886e-4	0.4	5	42
alpha [0.05,0.45,0.50]	[0.05, 0.45, 0.50]	2.0	48	8.886e-4	0.4	5	42
alpha [0.05,0.50,0.45]	[0.05, 0.50, 0.45]	2.0	48	8.886e-4	0.4	5	42
alpha [0.041,0.409,0.55]	[0.041,0.409,0.55]	2.0	48	8.886e-4	0.4	5	42
gamma 1.0	[0.05, 0.45, 0.60]	1.0	48	8.886e-4	0.4	5	42
gamma 3.0	[0.05, 0.45, 0.60]	3.0	48	8.886e-4	0.4	5	42
gamma 5.0	[0.05, 0.45, 0.60]	5.0	48	8.886e-4	0.4	5	42
hidden 32	[0.05, 0.45, 0.60]	2.0	32	8.886e-4	0.4	5	42
hidden 64	[0.05, 0.45, 0.60]	2.0	64	8.886e-4	0.4	5	42

hidden 96	[0.05, 0.45, 0.60]	2.0	96	8.886e-4	0.4	5	42
lr 5e-4	[0.05, 0.45, 0.60]	2.0	48	5.0e-4	0.4	5	42
lr 1e-3	[0.05, 0.45, 0.60]	2.0	48	1.0e-3	0.4	5	42
lr 2e-3	[0.05, 0.45, 0.60]	2.0	48	2.0e-3	0.4	5	42
dropout 0.2	[0.05, 0.45, 0.60]	2.0	48	8.886e-4	0.2	5	42
dropout 0.3	[0.05, 0.45, 0.60]	2.0	48	8.886e-4	0.3	5	42
dropout 0.5	[0.05, 0.45, 0.60]	2.0	48	8.886e-4	0.5	5	42
seq_len 3	[0.05, 0.45, 0.60]	2.0	48	8.886e-4	0.4	3	42
seq_len 7	[0.05, 0.45, 0.60]	2.0	48	8.886e-4	0.4	7	42
seed 7	[0.05, 0.45, 0.60]	2.0	48	8.886e-4	0.4	5	7
seed 123	[0.05, 0.45, 0.60]	2.0	48	8.886e-4	0.4	5	123

Optimal Configuration

hidden_dim	96
focal alpha	[0.05, 0.45, 0.60] (thick ice / thin ice / open water)
gamma	2.0
learning rate	8.886e-4
dropout	0.4
seq_len	5 (center segment and two neighboring segments on each side)
seed	42
Test mIoU	0.6978 (BiLSTM standalone, photon branch only)

Output Artifacts

- runs/bilstm/test_metrics.json : per-class IoU, mIoU, and macro F1
- runs/bilstm/confmat.png : row-normalized confusion matrix on T03CWT
- runs/bilstm/metrics.csv : epoch-level validation metrics
- archive/runs/lstm_sweep/sweep_results.csv : consolidated results for all 21 configurations

Stage 4: Fusion Model Training

The pretrained U-Net optical branch and BiLSTM photon branch are combined into a joint multimodal classifier. Three integration strategies are evaluated: Deep Fusion (primary contribution), Late Fusion, and Hybrid Fusion. Deep Fusion operates entirely at the deep feature level and achieves the highest test-set performance.

Required Inputs

outputs/	128×128 RGB patches from Stage 1
outputs_segmented/	Three-class ground-truth masks from Stage 2
runs/bilstm/ checkpoint	Pretrained BiLSTM weights from the Stage 3 optimal configuration (hidden_dim = 96)
Train / test partition	Training set: tiles T02CNA and T02CNC. Evaluation set: tile T03CWT (geographically disjoint held-out tile).

Implementation Notebooks

deep_fusion.ipynb	Primary contribution. Modalities are fused at the deep feature level via channel projection, spatial broadcast, and Squeeze-and-Excitation recalibration.
fusion_late_unet.ipynb	Decision-level fusion. Softmax outputs from each branch are averaged pixel-wise at inference time. The two branches are trained independently.
fusion_hybrid_unet.ipynb	Mid-network and decision-level fusion. Photon embeddings are injected into the U-Net decoder at an intermediate spatial scale; decision-level averaging is also retained.

Deep Fusion Construction Procedure

- The pretrained BiLSTM checkpoint from the Stage 3 optimal configuration is loaded into the photon branch.
- The U-Net encoder (ResNet-18, pretrained on ImageNet) is initialized via segmentation-models-pytorch. The decoder produces a 16-channel feature map of shape (16, 128, 128).
- Fusion bridge: the photon embedding is projected from hidden_dim to 16 channels via a 1×1 convolution and broadcast spatially to fill the 128×128 grid.
- The broadcast photon features are concatenated with the U-Net decoder map, yielding a 32-channel tensor.
- A Squeeze-and-Excitation block recalibrates channel weights: global average pooling, FC(32 to 8), ReLU, FC(8 to 32), Sigmoid, and channel-wise scaling.
- A 1×1 convolution maps the 32 recalibrated channels to 3 class logits.
- Training proceeds for 30 epochs with the Adam optimizer. The U-Net head is updated at the base learning rate; the photon branch is fine-tuned at one-tenth of the base learning rate.

Differential Learning Rate Strategy

The photon branch enters fusion training with representations already optimized for the standalone classification task. A learning rate reduced to one-tenth of the base value allows the branch to adapt to the joint fusion context, adjusting its feature representations to complement those of the U-Net while retaining the photon-discrimination patterns acquired in Stage 3. At the full learning rate, the pretrained weights are overwritten too rapidly; at zero (frozen branch), the branch cannot specialize to the fusion setting. This ratio was validated against both boundary cases in the ablation variants (fusion_v2 through fusion_v4 in archive/).

Output Artifacts

runs/deep_fusion/test_metrics.json Per-class IoU (thick ice, thin ice, open water), mIoU, macro F1

runs/deep_fusion/confmat.png Row-normalized confusion matrix on T03CWT

runs/deep_fusion/loss_curve.png Training and validation loss curves

runs/deep_fusion/summary_vs_all.csv Side-by-side comparison with all baseline models

runs/bilstm/ Standalone photon-branch evaluation outputs (same format)

Reproduction Commands

```
# Deep Fusion (primary result)
jupyter nbconvert --to notebook --execute deep_fusion.ipynb

# Late Fusion
jupyter nbconvert --to notebook --execute fusion_late_unet.ipynb

# Hybrid Fusion
jupyter nbconvert --to notebook --execute fusion_hybrid_unet.ipynb

# Note: set CUDA_VISIBLE_DEVICES=0 before launching.
# Each 30-epoch run requires approximately 60-90 min on an RTX A6000.
```

Stage 5: Evaluation Metrics and Full Reproduction

All models are evaluated exclusively on the geographically held-out tile T03CWT. No patches from T03CWT appear in any training or validation split.

mIoU	Mean Intersection-over-Union across the three classes. Primary ranking metric.
Per-class IoU	IoU reported separately for thick ice, thin ice, and open water. Thin ice is the most challenging class across all evaluated models.
Macro F1	Unweighted average of per-class F1 scores. Complements mIoU by assigning equal weight to each class regardless of class frequency.
Confusion matrix	Row-normalized: each row corresponds to a true class and each column to a predicted class. Diagonal entries represent per-class recall. Stored as <code>confmat.png</code> for each experimental run.

Complete Pipeline Reproduction

```
# Stage 1: extract 128x128 patches
python crop_all.py

# Stage 2: generate three-class ground-truth masks
python segment_all.py

# Stage 3: BiLSTM hyperparameter sweep (21 configurations)
jupyter nbconvert --to notebook --execute lstm_sweep.ipynb

# Stage 4a: train Deep Fusion model (primary result)
jupyter nbconvert --to notebook --execute deep_fusion.ipynb

# Stage 4b: train Late Fusion model
jupyter nbconvert --to notebook --execute fusion_late_unet.ipynb

# Stage 4c: train Hybrid Fusion model
jupyter nbconvert --to notebook --execute fusion_hybrid_unet.ipynb
```

Repository Directory Reference

IS2_Corrected_data/	ICESat-2 ATL03 photon CSV files (input). Six files: two beams × three tiles.
S2_tiff/	Sentinel-2 GeoTIFF scenes (input). Not tracked by version control; supply locally.
outputs/	Extracted 128×128 RGB patches (Stage 1 output). Not tracked.
outputs_segmented/	Per-pixel ground-truth segmentation masks (Stage 2 output). Not tracked.
runs/bilstm/	BiLSTM optimal-configuration checkpoint and evaluation outputs (Stage 3).
runs/deep_fusion/	Deep Fusion training outputs and test metrics (Stage 4).

confusion_matrices/	Final row-normalized confusion matrices for all five models (green and blue colormaps).
archive/runs/lstm_sweep/	Per-configuration artifacts for all 21 sweep runs and sweep_results.csv summary.
archive/runs/	Archived outputs for ablation variants (fusion_v2, fusion_v3, fusion_v4) and earlier experimental runs.
papers/	Reference literature.

Computational Environment

Python	3.9 or later
GPU	CUDA-capable device with at least 10 GB VRAM (reported experiments used an NVIDIA RTX A6000)
Dependencies	torch, torchvision, segmentation-models-pytorch, numpy, pandas, pillow, matplotlib, scikit-learn, tqdm, jupyter, nbconvert
Installation	pip install -r requirements.txt

Implementation Notes

- Large datasets, trained model checkpoints (*.pt), and patch caches are excluded from version control via .gitignore and must be regenerated or supplied locally.
- Each notebook specifies input and output paths in a dedicated configuration cell near the top; these paths should be adjusted to reflect the local directory layout prior to execution.
- The notebook_output/ directory contains the ATL03 data preparation notebook used to construct the photon feature sequences.